

Esercizio 1 (10 punti) Realizzare una funzione `Diff(A,k)` che, dato un array `A[1,n]` ordinato in senso decrescente, verifica se esiste una coppia di indici `i, j` tali che $A[i] - A[j] = k$. Restituisce la coppia di indici se esiste e (0,0) altrimenti. La funzione non deve alterare l'input e deve operare in spazio costante. Scrivere lo pseudocodice, provarne la correttezza e valutarne la complessità.

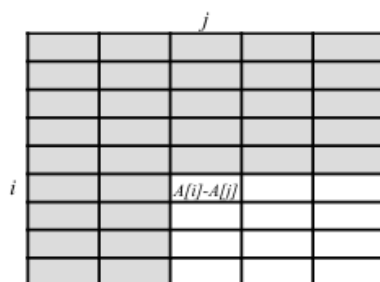
Soluzione:

Il codice può essere:

```
diff(A, n, k):
    i=1
    j=1
    while (i<=n) and (j<=n) and (A[i]- A[j] <> k)
        if (A[i]- A[j] < k)
            j++
        else
            i++

    if (i <= n) and (j<=n)
        return (i,j)
    else
        return (0,0)
```

È facile vedere che si mantiene l'invariante $\forall (i', j') \in [1, n]. (i' < i) \vee (j' < j) \Rightarrow A[i] - A[j] \neq k$, ovvero una coppia (i', j') tale che $A[i] - A[j] = k$ può esistere solo tra le coppie ancora esplorabili ($i' \geq i$ e $j' \geq j$), ovvero, graficamente nella parte non grigia:



Infatti, inizialmente, con $i = j = 1$, l'invariante è vacuamente vero.

Ad ogni iterazione, se entro nel ciclo, ci sono due possibilità:

- Se $A[i] - A[j] < k$, allora incremento j . In questo modo, escludo dall'esplorazione le coppie (i', j) con $i' \geq i$ per le quali, dato che l'array è decrescente e quindi $A[i] \geq A[i']$, vale

$$A[i'] - A[j] \leq A[i] - A[j] < k.$$

Dunque non escludo coppie utili, l'invariante continua a valere.

- Dualmente, $A[i] - A[j] > k$, allora incremento i . In questo modo, escludo dall'esplorazione le coppie (i, j') con $j' \geq j$ per le quali, dato che l'array è decrescente e quindi $A[j] \geq A[j']$, vale

$$A[i] - A[j'] \geq A[i] - A[j] > k.$$

Dunque, anche in questo caso, non escludo coppie utili, l'invariante continua a valere.

Quando esco dal ciclo, se $A[i] - A[j] = k$, ho concluso con successo. Altrimenti deve essere $i > n$ o $j > n$, che unitamente all'invariante, mi permettono di concludere che per ogni $i, j \in [1, n]$, $A[i] - A[j] \neq k$, come desiderato.

Da questo la correttezza segue immediatamente. La complessità è lineare. Il numero di iterazioni è pari al più a $2n - 1$, dato che i e j partono da 1, sono limitate da n ed ogni iterazione aumenta una delle due. Dato che ciascuna iterazione ha costo costante, ottengo $T(n) = O(2n - 1) = O(n)$.

Esercizio 1 (10 punti) Un array di interi $A[1..n]$ si dice *3-ordinato* se per ogni coppia di indici $i, j \in [1, n]$ con $i \leq j$ vale $A[i] \% 3 \leq A[j] \% 3$, dove $k \% 3$ indica il resto della divisione intera di k per 3. Realizzare una funzione `3Order(A)` che dato un array $A[1..n]$, lo rende 3-ordinato. Valutarne la complessità in tempo e in spazio, e indicare se l'algoritmo è stabile. Anche qualora si usasse un algoritmo di ordinamento noto, lo pseudo-codice va comunque scritto esplicitamente.

Soluzione: Un'osservazione semplice ma fondamentale è che di fatto si vuole ordinare l'array ottenuto sostituendo ogni valore $A[i]$ con $A[i] \% 3$.

Si può dunque ricorrere un algoritmo di ordinamento classico, sostituendo, nel confronto, i valori $A[i]$ con $A[i] \% 3$, ovvero con il loro resto modulo 3.

Ad esempio, se si usa il `Mergesort` si avrà complessità di tempo $O(n \log n)$ e spazio $O(n)$, e l'algoritmo è stabile:

```

3Order (A, p, r)
    if p < r
        q = (p+r)/2
        3Order (A, p, q)
        3Order (A, q+1, r)
        Merge (A, p, q, r)

Merge (A, p, q, r)
    n1 = q-p+1
    n2 = r-q
    for i = 1 to n1
        L[i] = A[i]
    for j = 1 to n2
        R[j] = A[p+j]

    L[n1+1]=R[n2+1] = infinity

    i=p
    j=q+1
    for k = p to r
        if (A[i]%3 <= B[j]%3): // si assume infinity%3 = infinity
            A[k] = L[i]
            i++
        else
            A[k]=B[j]
            j++
    }

```

Tuttavia si può osservare che $A[i]\%3$ ha valori in $\{0,1,2\}$ così che si può facilmente utilizzare un counting sort, con complessità di tempo $O(n)$ e spazio $O(n)$, stabile.

```
3Order (A, B, n)
    allocate C[0..2]

    for i=0 to 2
        C[i]=0

    for j=1 to n
        C[A[j]]++

    for i=1 to 2
        C[i]=C[i-1]+C[i]

    for j=n downto 1
        B[C[A[j]]] = A[j]
        C[A[j]]--
```

La soluzione migliore è probabilmente operare una tripartizione, come visto per il QuickSort, sulla base del valore di $A[i]\%3$. Concretamente si scorre l'array con un indice k mantenendo tre partizioni, corrispondenti ai valori 0, 1 e 2 per $A[i]\%3$: valore 0 in $A[1..p-1]$, valore 1 in $A[p..k-1]$, valore 2 in $A[j..n]$. La complessità di tempo è ancora $O(n)$ ma l'algoritmo è in place, quindi spazio $O(1)$. L'algoritmo non è stabile.

```
3Order (A,n)
    i = 0
    k = 1
    j = n+1
    while k < j

        if A[k]%3 == 0
            i++
            A[k] <-> A[i]
            k++

        else if A[k]%3 == 1
            k++

        else // A[k]%3 == 2
            j--
            A[k] <-> A[j]
```

Esercizio 1 (9 punti) Realizzare una funzione booleana `checkSum(A,B,C,n)` che dati tre array $A[1..n]$, $B[1..n]$ e $C[1..n]$ con valori numerici, verifica se esistono tre indici i, j, k con $1 \leq i, j, k \leq n$ tali che $A[i] + B[j] = C[k]$. Valutarne la complessità.

Soluzione: L'idea consiste nell'ordinare i vettori B e C e a questo punto verificare per ogni elemento $A[i]$ del primo array se ne esiste uno del secondo $B[j]$ la cui somma produce un elemento $C[k]$ del terzo. Il fatto che gli array B e C siano ordinati permette di scorrerli una sola volta per ogni elemento del primo array.

```
checkSum(A,B,C,n):
    # ordina B e C
    Sort(B)
    Sort(C)

    i = 1
    found = false

    while (i <= n) and not found
        # per ogni A[i] verifica se per qualche elemento B[j] vale che
        # A[i]+B[j]=C[k] scorrendo B e C

        j = k = 1
        while (j <= n) and (k <= n) and (A[i] + B[j] <> C[k])
            if (A[i] + B[j] < C[k]):
                j++
            else
                k++
        if (j<=n and k<=n)
            found = true
        else
            i++

    return found
```

L'invariante del ciclo esterno è che se `found` è falsa, allora per ogni $i' < i$, e qualunque siano j' e k' , vale $A[i'] + B[j'] \neq C[k']$. Se invece `found` è vera, allora $i, j, n \leq k$ e $A[i] + B[j] = C[k]$.

Per il ciclo interno, abbiamo che per ogni $j' < j$ e per ogni $k' < k$ vale $A[i] + B[j'] \neq C[k']$. Per il mantenimento dell'invariante, unico fatto da osservare è che quando $A[i] + B[j] < C[k]$ posso incrementare j mantenendo l'invariante. Infatti so che per ogni $j' < j$, dato che B è ordinato, vale $B[j'] \leq B[j]$ e quindi $A[i] + B[j'] \leq A[i] + B[j] < C[k]$. Il ragionamento quando $A[i] + B[j] > C[k]$ è incremento k è duale.

La complessità è $O(n^2)$ dato che ho due cicli annidati. In quello esterno, quando non esco i aumenta, quindi il numero di iterazioni è limitato da n . In quello interno, quando non esco, j oppure k aumentano, quindi il numero di iterazioni è limitato da $2n$. Complessivamente, nel caso peggiore, ho $n * 2n = 2n^2 = \Theta(n^2)$ iterazioni, ciascuna di costo costante.

A questo si somma il costo degli ordinamenti, che però posso realizzare in tempo $\Theta(n \log n)$ di modo che non incida sulla complessità asintotica.

Esercizio 1 (8 punti) Realizzare una funzione booleana `triple(A)` che dato un array di numeri $A[1..n]$ verifica se esistono tre indici (non necessariamente diversi) i, j e k , tali che $A[i] + A[j] = A[k]$ e restituisce un corrispondente valore booleano. Ad esempio per $[85, 10, 29, 93, 95]$ restituisce `true`, dato che $85 + 10 = 95$, mentre per $[70, 45, 83, 21, 68]$ restituisce `false`. Si suggerisce di ordinare l'array. Valutare la complessità.

Soluzione: Assumiamo inizialmente che l'array A sia ordinato. In questo caso, si può cercare, per ogni indice k , una coppia (i, j) tale che $A[k] = A[i] + A[j]$.

```
# assume A[1,n] ordinato
```

```
# cerca tre indici i,j,k tali che A[i]+A[j]=A[k]
```

```
Triple(A)
```

```
    n = A.length
```

```
    found = false
```

```
    k=1
```

```
    while (not found) and (k <= n)
```

```
        # cerca se ci sono due indici tali che A[i]+A[j]= A[k]
```

```
        i=1
```

```
        j=n
```

```
        # invariante:
```

```
        #   per ogni  $i' < i$ , e  $j'$  qualunque  $A[i'] + A[j'] < A[k]$ 
```

```
        #   per ogni  $j' > j$ , e  $i'$  qualunque  $A[i'] + A[j'] < A[k]$ 
```

```
        while (i <= j) and (A[i]+A[j] <> A[k])
```

```
            if A[i]+A[j] < A[k]
```

```
                i = i+1
```

```
            else
```

```
                j = j-1
```

```
        found = i <= j
```

```
        k=k+1
```

```
    return found
```

La complessità è $O(n^2)$: infatti, il ciclo interno, ad ogni iterazione incrementa i o decrementa j , quindi il numero massimo di iterazioni è n , e viene ripetuto al più n volte per effetto del ciclo esterno.

Se l'array non è ordinato, si può ordinarlo preventivamente, cosa che si può fare con costo $O(n \log n)$ e quindi senza alterare la complessità asintotica complessiva.

Esercizio 1 (10 punti) Un array di interi $A[1..n]$ si dice *triangolare* se esiste $q \in [1, n]$ tale che $A[1..q]$ è ordinato in modo crescente e $A[q..n]$ è ordinato in modo decrescente. Realizzare una funzione $\text{maxTr}(A)$ che dato un array triangolare $A[1..n]$, senza elementi ripetuti, ne trova il massimo. Valutarne la complessità.

Soluzione: Si realizza una procedura divide et impera che opera sul sotto-array $A[p, r]$, che inizialmente sarà l'intero array. Se $p = r$, ovvero il sotto-array ha dimensione 1, semplicemente si ritorna l'unico elemento $A[p]$. Altrimenti, si considera il punto intermedio $q = \lfloor p + r \rfloor$. Se $A[q] < A[q + 1]$ significa che la parte $A[p..q]$ è crescente e la parte $A[q+1..r]$ è ancora triangolare, per cui il massimo si troverà in quest'ultima e può essere cercato ricorsivamente. Dualmente se $A[q] > A[q + 1]$ significa che la parte $A[q+1..r]$ è decrescente e la parte $A[p..q]$ è ancora triangolare, per cui il massimo si troverà in quest'ultima.

```
maxTr(A,p,r)
  if p=r
    return A[p]
  else
    q = (p+r)/2
    if A[q]<A[q+1]
      return maxTr(A,q+1,r)
    else
      return maxTr(A,p,q)

maxTr(A)
  return maxTr(A,1,A.length)
```

La complessità è espressa dalla ricorrenza

$$T(n) = T(n/2) + c$$

Si può risolvere utilizzando il master theorem, confrontando $n^{\log_b a} = n^{\log_2^2} = n$ con $f(n) = c$. Dato che $f(n) = O(n^{\log_b a - \epsilon}) = O(n^{1-\epsilon})$ per $0 < \epsilon < 1$ si conclude che siamo nel primo caso del teorema che ci dà la soluzione $T(n) = \Theta(\log n)$.